
Forecasting Fantasy Football Player Performance with Transformer Sequence Models

Tyler Williams
M.S. Computer Engineering
NC State University
tdwill122@ncsu.edu

1 Introduction

Fantasy football is a predictive game in which participants draft real NFL athletes and earn weekly points based on players’ on-field performance. Lineup choices rely on accurate projections, and, with more than 29 million participants in the United States representing a multi-billion-dollar industry [2], improved forecasting has significant strategic and economic impact.

Existing Approach. Major platforms such as ESPN, Yahoo, and Sleeper primarily depend on expert-adjusted statistical models, matchup heuristics, simple trend extrapolation, and manual overrides. These methods often suffer from analyst bias, limited modeling of nonlinear interactions, and insufficient use of time-series context. Player usage (e.g. targets, carries, snap share, and route participation) evolves across weeks, yet is only partially incorporated in traditional systems.

Motivation. Player performance forms a natural temporal sequence. Transformers excel at modeling long-range dependencies, nonlinear interactions, and heterogeneous context. Recent success of sequence models in domains such as language modeling and multivariate forecasting motivates their application to fantasy sports.[3]

Challenges. Real NFL data is noisy and sparse, with missing values, inconsistent schema, unpredictable injuries, and context shifts due to trades or coaching changes. Constructing consistent player-week sequences and encoding temporal context requires careful engineering.

Contributions.

- A complete pipeline for acquiring and structuring multi-season NFL data using `nflreadpy`. [4]
- A sequence construction method using 10-week rolling windows with lag, moving averages, opponent defensive features, and team context.
- A transformer encoder architecture adapted for multivariate time-series with categorical embeddings and a fused representation.
- Empirical evaluation showing the transformer slightly outperforms Yahoo’s published weekly player projections for the 2025 season.
- Code stored in GitHub: <https://github.com/twilliams235/fantasy-football-oracle>

2 Method

2.1 Data Processing and Feature Construction

Using `nflreadpy`, I collect weekly statistics for all NFL skill-position players (QB, RB, WR, TE) from 2017–2025. Each row in the dataset represents a *player-week*. After cleaning inconsistent

schema fields, I compute PPR fantasy scoring either directly or using the standard formula:

$$\text{PPR} = \text{rec} + \frac{\text{rush_yds}}{10} + \frac{\text{rec_yds}}{10} + \frac{\text{pass_yds}}{25} + 6(\text{rush_td} + \text{rec_td}) + 4\text{pass_td} - 2\text{int}. \quad (1)$$

For each player-week, the prediction target is the *next week's* PPR fantasy points. To capture short-, medium-, and long-term tendencies, I compute lagged values (1 game), 3-game moving averages, and 5-game moving averages for key stats: fantasy points, targets, carries, receptions, and yards. Context features include:

- Team and opponent identifiers (categorical).
- Opponent defensive fantasy points allowed by position.
- Team-level pass rate for that week.

2.2 Sequence Construction

For each player, I sort all weekly records chronologically and gather a rolling history of up to the 10 most recent games. Early-season sequences are padded at the front. Each sequence contains:

- A $(10 \times d)$ numeric matrix of normalized features.
- A (10×3) categorical matrix (team ID, opponent ID, position ID).
- A binary mask indicating valid weeks vs. padding.

All model-ready inputs (e.g. numeric sequences, categorical embeddings, masks, and labels) are stored compactly inside `player_sequences_npz.npz`, which is the file the training and evaluation pipelines load directly when constructing batches.

2.3 Model Architecture

The model is based on a transformer encoder with modifications for multivariate forecasting. The numeric vector for each week is projected to a $d_{\text{model}} = 192$ representation. Three embedding tables map team, opponent, and position indices to dense vectors. All embeddings are concatenated and fused with a linear layer. Positional encodings represent the within-sequence time order.

A learnable [CLS] token is prepended to the sequence. Three transformer encoder layers (4 heads, GELU activation) capture interactions between context, historical performance, and player state. The final prediction head takes the [CLS] token, optionally concatenated with the position embedding of the most recent week, and outputs a single scalar representing the predicted next-week PPR score.

2.4 Training Procedure

Training uses seasons 2017–2024, while all of 2025 is held out for evaluation. Batch size is 256, and optimization uses AdamW with weight decay 10^{-4} . A warmup+cosine decay schedule controls learning rate, beginning at $2 \cdot 10^{-4}$. Loss is Smooth L1 (Huber), and gradient norm is clipped to 1.0. Mixed-precision training (AMP) accelerates computation.

3 Results

The primary metric is mean absolute error (MAE) on 2025 predictions. I compare model outputs directly with Yahoo’s public weekly projections for the same set of player-weeks.

Model	MAE (2025 Season)
Yahoo Projections	6.660
Transformer	6.516

Table 1: Comparison of next-week fantasy point MAE on all 2025 player-weeks.

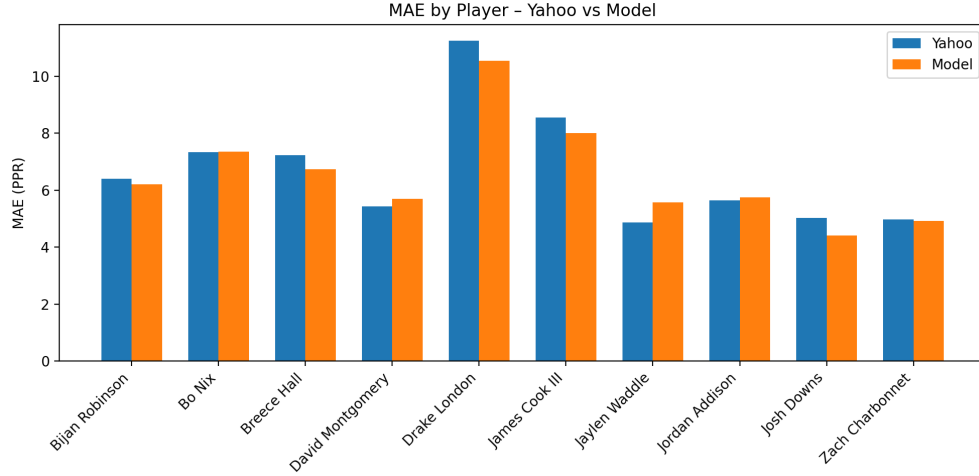


Figure 1: Per-player MAE comparison between Yahoo and the transformer model (weeks 1–10, 2025).

Figure 1 shows per-player MAE for a set of fantasy-relevant players across weeks 1–10 of 2025. The transformer is competitive with, and often slightly better than, Yahoo on most players.

Figure 2 plots the distribution of prediction errors (predicted minus actual PPR) across all 2025 validation samples. Most errors cluster near zero with a slight left tail (occasional large over-performances), reflecting the inherent volatility of NFL performance.

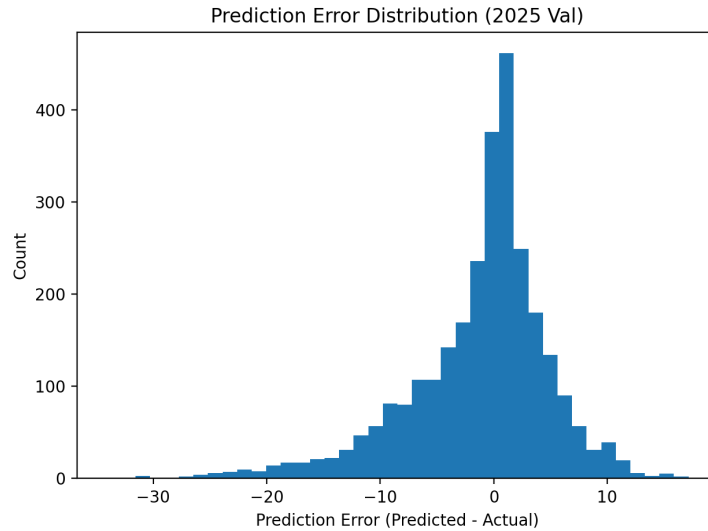


Figure 2: Distribution of transformer prediction errors on 2025 validation data.

4 Related Work

Prior NFL forecasting efforts often use generalized linear models, boosted trees, or expert systems combining injury news and heuristics [1]. Fantasy analytics communities have evaluated projection bias across platforms but typically rely on traditional models or simple aggregations. Transformers have been successfully applied to multivariate time-series forecasting (e.g., financial and energy data) but have not been widely used for fantasy sports. This work provides an initial demonstration of their suitability for structured sports prediction.

5 Code Snippets and Experiment Details

The full training pipeline includes sequence building, normalization, categorical encoding, model training, and evaluation. The core experiment can be reproduced with the following commands:

```
# 1) Build sequences from raw nflreadpy data
python make_player_sequences.py

# 2) Train the transformer (2017-2024 train, 2025 val)
python train.py

# 3) Generate plots and aggregate metrics for 2025
python make_performance_plots.py

# 4) Show the evaluation performance of a single player
python predict_players.py --name [NAME]
```

Within the model, the forward pass fuses numeric and categorical features and applies the transformer encoder:

```
h_num = self.num_proj(x_num) # [B, T, d_model]
cat_embs = [emb(x_cat[:, :, i]) for i, emb in enumerate(self.embs)]
h = torch.cat([h_num] + cat_embs, dim=-1) # [B, T, d_model*(1+num_cat)]
h = self.fuse(h)
if self.use_pos:
    pos_ids = torch.arange(T, device=device).unsqueeze(0).expand(B, T)
    h = h + self.pos_enc(pos_ids)
cls_tok = self.cls.expand(B, 1, -1)
z = self.encoder(torch.cat([cls_tok, h], dim=1),
                  src_key_padding_mask=key_padding)
yhat = self.head(z[:, 0, :]).squeeze(-1)
```

Evaluation uses MAE over the 2025 hold-out season:

```
with torch.no_grad():
    model.eval()
    abs_err = 0.0; n = 0
    for batch in val_loader:
        yhat = model(batch["x_num"], batch["x_cat"], batch["mask"])
        abs_err += (yhat - batch["y"]).abs().sum().item()
        n += batch["y"].numel()
    mae_2025 = abs_err / n
```

References

- [1] John Brooks and Erik Strumbelj. A quantitative evaluation of nfl game predictions from statistical models and sportsbooks. *Journal of Quantitative Analysis in Sports*, 2016.
- [2] Steve Gardner. Money, power, women: The driving forces behind fantasy football’s skyrocketing popularity. *USA Today*, 2023.
- [3] Bryan Lim and Stefan Zohren. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 2021.
- [4] nflverse contributors. nflreadpy: Python access to nfl play-by-play and player statistics. <https://github.com/nflverse/nflreadr>, 2024.

NeurIPS Paper Checklist

1. **Claims**
Answer: [Yes] *Justification:* The abstract and introduction state scope (player-level weekly PPR forecasting) and contributions (pipeline, transformer model, preliminary results, plan).
2. **Limitations**
Answer: [Yes] *Justification:* We discuss rookies/cold-start, injury shocks, and drift; present mitigations and remaining risks.
3. **Theory Assumptions and Proofs**
Answer: [NA] *Justification:* Empirical modeling paper; no novel theoretical results.
4. **Experimental Result Reproducibility**
Answer: [Yes] *Justification:* Code, seeds, splits, features, and training recipe are specified; checkpointing provided.
5. **Open access to data and code**
Answer: [Yes] *Justification:* Data derived from nfl-data-py; scripts documented. (Public release contingent on course policy.)
6. **Experimental Setting/Details**
Answer: [Yes] *Justification:* Optimizer, loss, schedule, batch sizes, and model dimensions are stated.
7. **Experiment Statistical Significance**
Answer: [NA] *Justification:* Milestone report; we will add CIs/variability across runs in the final submission.
8. **Experiments Compute Resources**
Answer: [Yes] *Justification:* Single-GPU/CPU compatible; AMP enabled; storage paths specified. We will tabulate run-time in the final.
9. **Code Of Ethics**
Answer: [Yes] *Justification:* Sports forecasting on public data with minimal risk; we adhere to usage terms and course guidelines.
10. **Broader Impacts**
Answer: [Yes] *Justification:* Positive: better decision support and methods for time-series. Negative: potential gaming of fantasy markets; addressed via transparent reporting.
11. **Safeguards**
Answer: [NA] *Justification:* No high-risk assets; no release of sensitive data.
12. **Licenses for existing assets**
Answer: [Yes] *Justification:* We cite nfl-data-py and respect its license; any released code will include license metadata.
13. **New Assets**
Answer: [NA] *Justification:* No new dataset/model release beyond coursework artifacts at milestone.